NVIDIA Unreal Engine DLSS Super Resolution / DLAA Plugin

Table of Contents

- Overview
  - DLSS 4 and the NVIDIA Unreal Engine DLSS Super Resolution Plugin
  - What's documented in this file
  - What's available and documented elsewhere
- Quickstart
- Plugins and Platforms
  - Windows System requirements
  - Plugins and other Platforms
  - XBOX and UE WinGDK Platform Support
    - Using the WinGDK Extension Plugin
  - Plugins and Custom Engines
- Troubleshooting
  - Diagnosing DLSS Issues in the Editor
  - Incompatibilities with API Capture Tools such as RenderDoc
  - Incompatibilities with Depth of Field
  - Incompatibilities with the Mobile Forward Renderer
  - Incompatibilities with Non-Cooker Commandlets
  - Verify Engine side plugin hooks
  - Enabling NGX DLSS Logging on End User machines
  - Enabling NGX DLSS Logging during Development
  - DLSS On-Screen Indicator
  - Compatibility with other third party plugins
- Command Line Options And Console Variables and Commands
  - Enabling DLSS (Engine Side)
  - Enabling Motion vectors for DLSS
  - Enabling Bias Current Color Mask for DLSS
  - Enabling DLSS-SR/DLAA (Plugin Side)
  - Enabling DLSS-RR (Plugin Side)
  - DLSS Runtime Image Quality Tweaks
  - DLSS Binaries
  - DLSS memory usage
  - DLSS presets
  - NGX Project ID
  - Multi GPU Support (Experimental)
  - Miscellaneous
- DLSS and the NIS NVIDIA Image Scaling plugin
- DLSS/DLAA in the Editor
  - Enabling DLSS/DLAA for a project
  - Enabling DLSS/DLAA in Level Editor Viewports
  - DLSS Plugin Settings
  - Editor Settings
  - DLSS Blueprints
  - DLSS Movie Render Queue Support
- Distributing DLSS
- Upgrading from DLSS plugin versions for earlier engines
  - Screen percentage
- Performance benefits of DLSS-SR
- Performance benefits of DLSS-RR
- DLSS API and UI Documentation
- Graphics Debug and Validation Layers
  - UE 5.6 and newer
  - UE 5.5 and older
  - Experimental Backports from 5.6 `ID3D12DynamicRHI` methods to 5.5 and older

Overview

DLSS 4 and the NVIDIA Unreal Engine DLSS Super Resolution Plugin

The NVIDIA *DLSS Super Resolution/Ray Reconstruction/DLAA* plugin is part of a wider suite of related NVIDIA performance and

image quality improving technologies and corresponding NVIDIA Unreal Engine plugins:

- NVIDIA DLSS *Frame Generation (DLSS-FG)* boosts frame rates by using AI to render additional frames. *DLSS-FG* requires a GeForce RTX 40 series graphics card.
- NVIDIA DLSS *Multi Frame Generation (DLSS-MFG)* uses AI to boost frame rates by generating up to 3 additional high-quality frames, all while optimizing responsiveness with NVIDIA Reflex. DLSS Multi Frame Generation is available for GeForce RTX 50 Series GPUs.
- NVIDIA DLSS *Super Resolution (DLSS-SR)* boosts frame rates by rendering fewer pixels and using AI to output high resolution frames. *DLSS-SR* requires an NVIDIA RTX graphics card.
- NVIDIA DLSS *Ray Reconstruction (DLSS-RR)* uses AI to enhance image quality and generate additional pixels for intensive ray-traced scenes. Ray Reconstruction replaces hand-tuned denoisers with an NVIDIA supercomputer-trained AI network that generates higher-quality pixels in between sampled rays. This feature is available for all RTX GPUs and requires Super Resolution to be enabled.
- NVIDIA *Deep Learning Anti Aliasing (DLAA)* is used to improve image quality. *DLAA* requires an NVIDIA RTX graphics card.
- NVIDIA *Image Scaling (NIS)* provides best-in class upscaling and sharpening for non-RTX GPUs, both NVIDIA or 3rd party. Please refer to the NVIDIA *Image Scaling* Unreal Engine plugin for further details.
- NVIDIA *RTX Dynamic Vibrance* enhances visual clarity by dynamically tuning color saturation and contrast.
- NVIDIA *Reflex Low Latency* technology, synchronizes the CPU frame submission with GPU processing for just in time rendering.

NVIDIA *DLSS 4* combines DLSS Multi Frame Generation, DLSS Frame Generation, DLSS Super Resolution, DLSS Ray Reconstruction, NVIDIA DLAA, NVIDIA Reflex Low Latency and delivers boosted frame rates, great responsiveness, and better than native IQ.

What's documented in this file

The NVIDIA Unreal Engine DLSS-SR plugin provides:

- DLSS Super Resolution (DLSS-SR)
- Deep Learning Anti-Aliasing (DLAA)
- DLSS Ray Reconstruction (DLSS-RR)
- Movie Render Queue support for DLSS-SR,DLSS-RR, DLAA

What's available and documented elsewhere

The NVIDIA Unreal Engine Streamline plugins provide:

- DLSS Frame Generation (also called DLSS-G or DLSS-FG)
- DLSS Multi Frame Generation (also called DLSS-MFG)
- NVIDIA Reflex Low latency
- NVIDIA RTX Dynamic Vibrance (also called DeepDVC)

The NVIDIA Unreal Engine NIS plugin provides:

- NVIDIA Image Scaling

Quickstart

Please refer to the relevant section in this document for additional details.

1. Enable the *NVIDIA DLSS Super Resolution/Ray Reconstruction/DLAA* plugin in the Editor, then restart the editor.
   1. Separately, if you require support for the Movie Render Queue, ensure that the *Movie Render Queue DLSS/DLAA Support* plugin is enabled.
2. DLSS-SR/DLSS-RR/DLAA in the Editor: enable the following settings in the Project Plugin settings
   1. Enable DLSS to be turned on in Editor viewports (it is off by default)
   2. In the Viewport Options (triple-bar ≡ menu in the top left corner), set the Screen Percentage option to control the upscale percentage. Not all screen percentages are supported. Values in the range 50-67 are recommended for DLSS, 100 for DLAA, and 50-100 for DLSS-RR.
3. DLSS-SR/DLSS-RR/DLAA in Blueprint:
   1. The `EnableDLSS` DLSS blueprint library function provides a convenient method for setting the console variables for DLSS Super Resolution. This is recommended to be used when integrating support into a project's user interface and settings.
   2. The `Enable DLSS-RR` DLSS blueprint library function provides a convenient method for setting the console variables for DLSS Ray Reconstruction. This is recommended to be used when integrating support into a project's user interface and settings.
      1. Due to an engine runtime assertion, the `r.Lumen.Reflections.BilateralFilter` feature can't safely be changed at runtime in engine versions 5.2 and 5.3. See note below.
   3. The `GetDlssModeInformation` DLSS blueprint library function can be used to query optimal screen percentage to use for given quality mode. The screen percentage can then be used with `r.ScreenPercentage` in an `ExecuteConsoleCommand` node.
4. DLSS-SR/DLSS-RR/DLAA in PIE: `Edit -> Editor Preferences -> Performance` turn off "Override game screen percentage settings with editor settings in PIE" (it is on by default)
5. DLSS-SR in Game: make sure that the following console variables are set to enable DLSS Super Resolution:
   1. r.NGX.Enable 1 (read-only, can be overridden on the command line with -ngxenable)

    2. r.NGX.DLSS.Enable 1
    3. r.ScreenPercentage 66.7
6. DLAA in Game: make sure that the following console variables are set to enable DLAA
    1. r.NGX.Enable 1 (read-only, can be overridden on the command line with -ngxenable)
    2. r.NGX.DLSS.Enable 1
    3. r.ScreenPercentage 100
7. DLSS-RR in Game: DLSS-RR requires all denoisers to be deactivated. This is most true when the project uses ray tracing. Make sure that the following cvars are set.
    1. r.NGX.DLSS.Denoisermode 1
    2. r.Lumen.Reflections.BilateralFilter 0 - *In engine versions 5.2 and 5.3, changing this setting at runtime via console or blueprints can lead to engine runtime assertions if the resolution later changes. It should be set only by changing a config file such as DefaultEngine.ini. So if you wish to re-enable r.Lumen.Reflections.BilateralFilter when DLSS-RR is disabled, it would require a config setting change and engine restart.*
    3. r.Lumen.Reflections.ScreenSpaceReconstruction 0
    4. r.lumen.Reflections.Temporal 0
    5. r.Shadow.Denoiser 0
8. Check the log for `LogDLSS: NVIDIA NGX DLSS supported 1`
9. (Optionally) Enable the DLSS on screen indicator in the bottom left of the screen via `DLSS\Source\ThirdParty\NGX\Utils\ngx_driver_onscreenindicator.reg` to verify that DLSS is active

Plugins and Platforms

Windows System requirements

- Windows 10, 64 bit
    - at least version v1709, Windows 10 Fall 2017 Creators Update 64-bit.
- NVIDIA GeForce Driver
    - Required: driver issued after March 3, 2022 (for instance 512.15)
- NVIDIA RTX GPU (GeForce, Titan or Quadro) with DLSS Super Resolution support
- UE project using either
    - Vulkan
    - DX11
    - DX12

Plugins and other Platforms

The Blueprint modules are supported for all platforms and are stubbed out and return `NotSupported` for the various`QuerySupport` methods. This allows cross platform UI and settings BP and code to compile on all platforms.

The actually platform specific modules (e.g. `NGXD3D12RHI`) have the `PlatformAllowList` set to the actually supported platforms, such as `Win64`.

XBOX and UE WinGDK Platform Support

For games that will be released on Xbox, the DLSS UE Plugin is accompanied by a UE platform extension that provides DLSS plugin support for the UE WinGDK platform.

This extension comes in the form of a `DLSS_WinGDK.plugin` plugin extension and adds support to the DLSS Plugin, version 4.0.0 or later.

The WinGDK Platform support files can be found in the `Platforms` folder adjacent to the Plugins folder.

Note: The DLSSMoviePipelineSupport and NIS plugin do not need a WinGDK plugin extension since they either are editor only, or are using platform independent layers of the engine.

Note that you **must** have the following pre-requisites

- Be a member of the ID@Xbox program and have console access.
- Apply and get approval for Unreal Engine Console Access.

Using the WinGDK Extension Plugin

If installing as an engine plugin:

- Copy the files anywhere under the `Engine\Platforms\WinGDK\Plugins` folder.

If installing as a project plugin:

- Copy the files anywhere under the project's `Platforms\WinGDK\Plugins` folder

Plugins and Custom Engines

The published UE plugins `EngineVersion` in the respective .uplugin files matching the engine they have been packaged for.

Using a plugin with a mismatched engine (e.g. using the UE 5.6 plugin package with UE 5.5), or with a custom engine that might have non-standard `EngineVersion` fields is not supported.

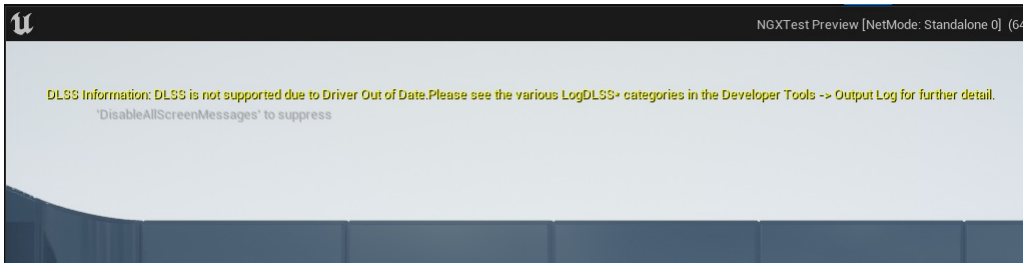
It will likely succeed to build, but then fail at cook time due to missing plugin Blueprint references.

The solution is to remove the `EngineVersion` field in the plugins when using with a custom engine.

Troubleshooting

Diagnosing DLSS Issues in the Editor

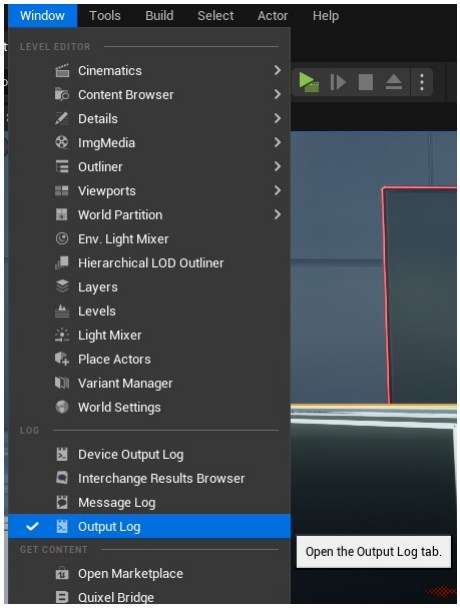
The DLSS plugin shows various common reasons why DLSS might not be working at the top of the screen (in non-Shipping build configurations). This message can also be turned off in the DLSS plugin settings, as discussed in the "DLSS plugin settings" section in this document.



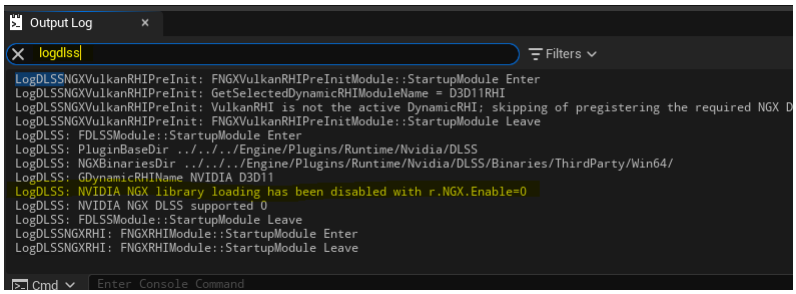
Additionally, the DLSS plugin modules write various information into the following UE log categories:

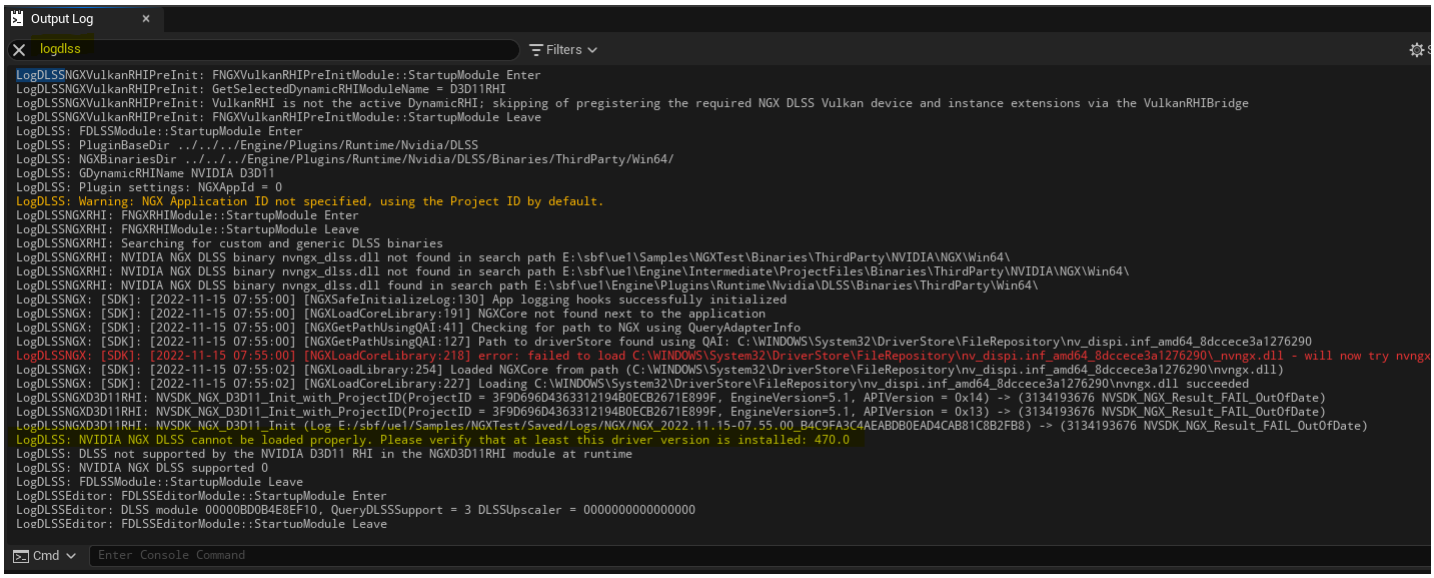
- LogDLSS
- LogDLSSEditor
- LogDLSSBlueprint
- LogDLSSNGXRHI
- LogDLSSNGXD3D11RHI
- LogDLSSNGXD3D12RHI
- LogDLSSNGXVulkanRHIPreInit
- LogDLSSNGXVulkanRHI
- LogDLSSNGX

Those can be accessed in the Editor under `Window -> Output Log`



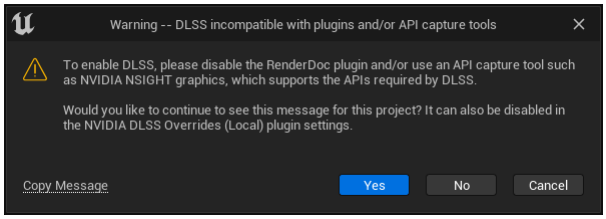
The Message log then can be filtered to show only the DLSS related messages to get more information on why DLSS might not be functioning as expected, as shown in these examples.





Incompatibilities with API Capture Tools such as RenderDoc

The Editor will show a warning at startup if DLSS incompatible API capture tools (such as RenderDoc) or plugins are used. To enable DLSS, please use an API capture tool such as NVIDIA NSIGHT Graphics, which support the NGX APIs required by DLSS.



Incompatibilities with Depth of Field

As DLSS workload occurs in the same spot as TAAU in the pipeline, visual differences in DoF are expected. To minimize the differences, it is recommended to use DLSS in 'Quality' or 'Ultra Quality' modes. It is possible to tweak the DoF settings in the camera actor to compensate for the differences. Please keep in mind this is content dependent. Thus for some scenes the differences can be minimal and avoided while in other cases it might be more challenging.

Incompatibilities with the Mobile Forward Renderer

This plugin is only compatible with the deferred renderer. The mobile renderer is not supported since it doesn't call the relevant `FSceneViewExtension` methods needed.

Incompatibilities with Non-Cooker Commandlets

This plugin disables the runtime components when run in commandlet (e.g. WorldPartitionBuilderCommandlet), regardless of whether -AllowRendering is specified or not.

Verify Engine side plugin hooks

The following cvars should be set to those values by default:

- r.TemporalAA.Upscaler 1
- r.Reflections.Denoiser 2

Enabling NGX DLSS Logging on End User machines

The DLSS plugin also pipes the NGX DLSS logs into the UE logging system into the `LogDLSSNGX` log category. It is enabled by

default and can be tweaked with the `r.NGX.LogLevel` console variable, or set with the -NGXLogLevel=X command line option.

This requires an NVIDIA GeForce **driver version 461.36** or later.

Enabling NGX DLSS Logging during Development

If r.NGX.EnableOtherLoggingSinks is set then additional NGX logging of the NVIDIA NGX software stack to files can be used as well, as discussed in the "NGX logging" chapter of the DLSS Programming Guide for details. The -NGXLogFileEnable and -NGXLogFileDisable command line options can also override the default setting. The DLSS SDK provides registry keys which can be set with the following .reg files which can be found in the plugin folder under `\DLSS\Source\ThirdParty\NGX\Utils\`:

- `ngx_log_on.reg`
- `ngx_log_off.reg`
- `ngx_log_verbose.reg`

The DLSS plugin will write those into subfolder under `$(ProjectDir)\Saved\Logs\` with a `NGX_$(TimeStamp)_$(GUID)` pattern

- `nvngx.log`
- `nvngx_dlss_2_1_34.log`
- `nvsdk_ngx.log`

DLSS On-Screen Indicator

The DLSS SDK provides registry keys which can be set with the following .reg files which can be found in the plugin folder under `\DLSS\Source\ThirdParty\NGX\Utils\`:

- `ngx_driver_onscreenindicator.reg`
- `ngx_driver_off_screenindicator.reg`

With the first registry key set, DLSS will display an indicator on-screen when it is enabled, enabling easier troubleshooting. The second registry key can be used to disable this indicator again.

Please see the DLSS Programming Guide for further details.

Compatibility with other third party plugins

The DLSS custom denoiser is meant to be chained with other denoisers. The DLSS denoiser will only be used when DLSS is active, otherwise it will pass to the next denoiser in the chain. But some other third party plugins that implement their own denoisers may not pass down the chain correctly. If you experience the wrong denoiser being used, you may need to work around this by manually setting GScreenSpaceDenoiser to the DLSS custom denoiser when DLSS is active. FDLSSModule::GetDLSSDenoiser() is provided to help with this.

Command Line Options And Console Variables and Commands

Enabling DLSS (Engine Side)

The DLSS plugin uses various engine side hooks, which can be configured by the following cvars. Their default values

- r.TemporalAA.Upscaler (1, default)
  - Enable a custom TAAU upscaling plugin, such as the DLSS plugin
- r.Reflections.Denoiser (2, default)
  - Enable a custom denoising plugin. The DLSS plugin makes use of this to improve image quality for raytraced reflections by adding additional TAA passes

Additionally, when using DLSS-RR it is strongly recommended to disable built-in denoising for Lumen reflections, using the setting `r.Lumen.Reflections.BilateralFilter=0`. In engine versions 5.2 and 5.3, changing this setting at runtime via console or blueprints can lead to engine runtime assertions if the resolution later changes. It should be set only by changing a config file such as DefaultEngine.ini. So if you wish to re-enable r.Lumen.Reflections.BilateralFilter when DLSS-RR is disabled, it would require a config setting change and engine restart.

Note: RTX UI Guidelines section

> When DLSS Super Resolution is turned on, make sure the anti-aliasing settings are disabled (both UI, as itself)

discusses the user interface and experiences. However **engine is required to run in TemporalAA/TSR mode** (r.AntiAliasingMethod 2/4 and r.TemporalAA.Upscaler 1) to allow the UE DLSS plugin to properly replace built-in TAA/TSR with DLSS-SR/RR and thus implement user experience recommended by the RTX UI Guidelines.

Enabling Motion vectors for DLSS

DLSS requires correct motion vectors to function properly. The following console variable can be used to render motion vectors for all

objects, and not just the ones with dynamic geometry. This can be useful if it's infeasible to e.g. change all meshes to stationary or dynamic.

- r.Velocity.ForceOutput (0, default)
  - Force the base pass to compute motion vector, regardless of FPrimitiveUniformShaderParameters.
  - 0: Disabled
  - 1: Enabled

Enabling Bias Current Color Mask for DLSS

DLSS may then “erase” a particular feature or can produce a “ghost” or trail behind a moving feature. If a problematic asset is discovered during testing, one option is to instruct DLSS to bias the incoming color buffer over the colors from previous frames.

To mark a component as a component that should be passed to the Bias Current Color buffer the Custom Depth rendering should be set to enabled and the stencil value of the component should be equal to Bias Current Color Mask Stencil Value (see below).

- r.NGX.DLSS.BiasCurrentColorMask (0, default)
  - Enable/Disable support for BiasCurrentColorMask.
  - 0: Disabled
  - 1: Enabled
- r.NGX.DLSS.BiasCurrentColorMaskStencilValue (-1,default)
  - The value that would be considered as Bias Color in the custom depth stencil buffer, Must not be set to 0!
  - -1: Use project settings value.
  - ≥1: Use CVar Value as stencil value.
  - **Note** Must be positive, non-zero.
  - **Note** Custom Depth Stencil must be set to Enabled With Stencil (r.CustomDepth=3).

Enabling DLSS-SR/DLAA (Plugin Side)

- r.NGX.Enable (1, default) read-only cvar, can also be overridden on the command line with **-ngxenable** and **-ngxdisable**
  - Whether the NGX library should be loaded. This allow to have the DLSS plugin enabled but avoiding potential incompatibilities by skipping the driver side NGX parts of DLSS.
- r.NGX.DLSS.Enable (1, default)
  - Enable/Disable DLSS-SR/DLAA.

There are two alternatives for presenting DLSS in a UI, either listing all DLSS modes explicitly or offering DLSS Off/Auto mode. For explicit DLSS modes, list all supported DLSS modes in the UI. Find the optimal screen percentage for the selected DLSS mode with `GetDLSSModeInformation`, set `r.ScreenPercentage` using that value, and call `EnableDLSS`. For DLSS auto mode, find the optimal screen percentage for the DLSS auto mode with `GetDLSSModeInformation`, if the screen percentage is not 0 then set `r.ScreenPercentage` using that value and call `EnableDLSS`.

The plugin includes two blueprint macros in the DLSSMacros asset for enabling DLSS and DLAA, one for explicit DLSS modes and one for DLSS auto mode. You can use a macro directly or copy it into your project to use as a starting point for your own work. You may need to enable "Show Engine Content" and "Show Plugin Content" in the content browser settings for the macros to be visible.

Blueprint functions:

- `Enable DLSS-SR`, `Is DLSS-SR Enabled`
- `Is DLSS-SR Supported`, `Query DLSS-SR Support`, `Get DLSS-SR Minimum Driver Version`, `Get Default DLSS Mode`
- `Is DLSS-SR Mode Supported`, `Get Supported DLSS-SR Modes`,`Get DLSS-SR Mode Information`, `Get DLSS-SR Screen Percentage Range`

Enabling DLSS-RR (Plugin Side)

- r.NGX.DLSS.DenoiserMode (0, default)
  - Configures how DLSS denoises
  - 0: off, no denoising (default)
  - 1: DLSS-RR enabled
- r.NGX.DLSS.BuiltInDenoiserOverride (-1, default)
  - Change what happens to the built-in denoisers
  - -1: automatic, depending on r.NGX.DLSS.DenoiserMode (default)
  - 0: skip all built-in denoising
  - 1: use built-in denoising

By default, when DLSS-RR is enabled it will override all built-in denoising, except `r.Lumen.Reflections.BilateralFilter`, due to an engine runtime assert in 5.2 and 5.3. For more details, please refer back to the Quick Start Section at the top of this guide.

Blueprint functions:

- `Enable DLSS-RR`, `Is DLSS-RR Enabled`
- `Is DLSS-RR Supported`, `Query DLSS-RR Support`, `Get DLSS-RR Minimum Driver Version`
- `Is RayTracing Available`
  - This bonus blueprint function helps us determine ray tracing availability to help populate and manage UI settings that would require ray tracing.
  - This function is already part of the `Is DLSS-RR Supported` query, but available separately for convenience.

DLSS Runtime Image Quality Tweaks

- r.NGX.DLSS.Reflections.TemporalAA (1, default)
  - Apply a temporal AA pass on the denoised reflections
- r.NGX.DLSS.WaterReflections.TemporalAA (1, default)
  - Apply a temporal AA pass on the denoised water reflections
- r.NGX.DLSS.EnableAutoExposure
  - 0: Use the engine-computed exposure value for input images to DLSS
  - 1: Enable DLSS internal auto-exposure instead of the application provided one - enabling this can alleviate effects such as ghosting in darker scenes (default)

DLSS Binaries

- r.NGX.BinarySearchOrder (0, default)
  - 0: automatic
    - use custom binaries from project and launch folder (*ProjectDir*)/*Binaries*/*ThirdParty*/*NVIDIA*/*NGX*/(Platform) if present
    - fallback to generic binaries from plugin folder
  - 1: force generic binaries from plugin folder, fail if not found
  - 2: force custom binaries from project or launch folder, fail if not found
  - 3: force generic development binaries from plugin folder, fail if not found. This is only supported in non-shipping build configurations

DLSS memory usage

- `stat DLSS`
  - shows how much GPU memory DLSS uses and how many DLSS features, i.e. instances of DLSS are allocated.
    - **Note**: This number might fluctuate a bit in UE5.5 and up due to the data collection interacts with the parallel RHI.
  - In steady state there should be 1 DLSS feature allocated per view. This value can increase temporarily, typically after changing the DLSS quality mode or resizing the window. This can be configured with the `r.NGX.FramesUntilFeatureDestruction` console variable

DLSS presets

The DLSS plugin offers render presets that allow tweaking different aspects of DLSS-SR/DLSS-RR upscaling. Usually there will be no need to change from the default presets. The DLSS presets are named by letters A,B,C.... and are described in more detail in the DLSS-SR Programming Guide and DLSS-RR Programming Guide

A particular preset can be forced separately for DLAA and for each DLSS quality mode in the DLSS plugin settings (For DLSS-SR: Edit -> Project Settings -> NVIDIA DLSS -> General Settings -> Advanced. For DLSS-RR: Edit → Project Settings → NVIDIA DLSS → DLSS-RR Settings → Advanced). Additionally, the DLSS preset can be globally overridden by setting the cvar `r.NGX.DLSS.Preset` and `r.NGX.DLSSRR.Preset` to a value from 0 to N (0=project setting, 1=A, 2=B, 3=C, ...).

Presets are separate for DLSS-SR and DLSS-RR in both the project settings as well as console variables.

DLSS presets are subject to changes such as deprecation with new versions of DLSS and via over the air (OTA) updates.

When a deprecated DLSS-Preset in a project setting is detected at startup, it will be reset to the default preset and a log message will be emitted. If running in the editor, a message box will additionally be shown. Additionally the respective enum values will be set as hidden.

To opt out of OTA updates, unselect the "Allow OTA Update" option in the DLSS plugin settings (Edit -> Project Settings -> NVIDIA DLSS -> General Settings). OTA updates can also be overridden on the command line with **-ngxdisableota** and **-ngxenableota**. The interaction of OTA updates with DLSS presets is described in more detail in the DLSS Programming Guide.

NGX Project ID

The DLSS plugin by default uses the project identifier to initialize NGX and DLSS. On rare occasion, NVIDIA might provide a special NVIDIA NGX application ID. The following console variable determines which one is used.

r.NGX.ProjectIdentifier (0, default)

- 0: automatic:
  - use NVIDIA NGX Application ID if non-zero, otherwise use UE Project ID
- 1: force UE Project ID

- 2: force NVIDIA NGX Application ID (set via the Project Settings -> NVIDIA DLSS plugin)

Please refer to the "Distributing DLSS" section for further details.

Multi GPU Support (Experimental)

The DLSS plugin supports multiple GPUs in certain circumstances, as shown in the following table. There AFR stands for Alternate-Frame-Rendering, i.e. SLI or CrossFire, and SFR stands for Split-Frame-Rendering.

RHI	AFR	SFR
D3D12RHI	no	no
D3D11RHI	yes	no
VulkanRHI	no	no

Notes:

- D3D12RHI
  - AFR is not supported
    - Primarily due to higher level renderer code not maintaining TAA (and thus DLSS) history across non-consecutive frames on the same GPU
  - SFR is not yet supported in 5.1 engines
- D3D11RHI
  - AFR is supported via driver based, automatic SLI support
- VulkanRHI
  - The VulkanRHI (as of UE 4.27) does not implement explicit MGPU, and thus neither AFR nor SFR are available

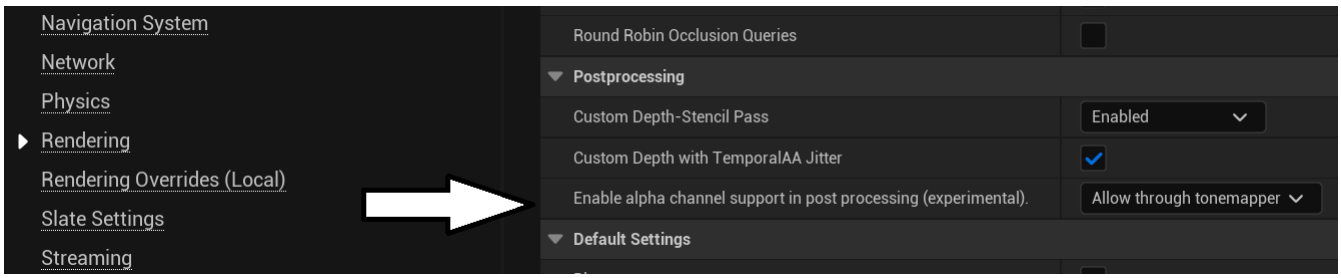
The following console variables can be used to adjust how DLSS interacts with the GPU nodes

- r.NGX.DLSS.FeatureCreationNode (-1, default)
  - Determines which GPU the DLSS feature is getting created on
  - -1: Create on the GPU the command list is getting executed on
  - 0: Create on GPU node 0
  - 1: Create on GPU node 1
- r.NGX.DLSS.FeatureVisibilityMask (-1, default)
  - Determines which GPU the DLSS feature is visible to
  - -1: Visible to the GPU the command list is getting executed on
  - 1: visible to GPU node 0
  - 2: visible to GPU node 1
  - 3: visible to GPU node 0 and GPU node 1

Miscellaneous

- r.NGX.Automation.Enable (0, default)
  - Enable automation for NGX DLSS image quality and performance evaluation.
- r.NGX.Automation.ViewIndex (0, default)
  - Select which view to use with NGX DLSS image quality and performance automation.
- r.NGX.Automation.NonGameViews (0,default)
  - Enable non-game views for NGX DLSS image quality and performance automation.
- r.NGX.FramesUntilFeatureDestruction (3, default)
  - Number of frames until an unused NGX feature gets destroyed
- r.NGX.DLSS.MinimumWindowsBuildVersion (16299, default for v1709)
  - Sets the minimum Windows 10 build version required to enable DLSS
- r.NGX.LogLevel (1, default)
  - Determines the minimal amount of logging the NGX implementation. Please refer to the DLSS plugin documentation on other ways to change the logging level.
  - 0: off
  - 1: on
  - 2: verbose
- r.NGX.RenameNGXLogSeverities (1, default)
  - Renames 'error' and 'warning' in messages returned by the NGX log callback to 'e_rror' and 'w_arning' before passing them to the UE log system
  - 0: off
  - 1: on, for select messages during initialization
  - 2: on, for all messages
- r.NGX.DLSS.ReleaseMemoryOnDelete (1, default)
  - Enable/disable releasing DLSS related memory on the NGX side when DLSS features get released
- r.NGX.DLSS.EnableAlphaUpscaling (-1, default)
  - Alpha channel upscaling
  - -1: based of r.PostProcessing.PropagateAlpha (default)
  - 0: off

- 1: on
- **Note** r.PostProcessing.PropagateAlpha MUST be enabled for this feature to work.
- **Note** Upscaling RGBA might have up to 20% pref cost compared to RGB, use wisely.



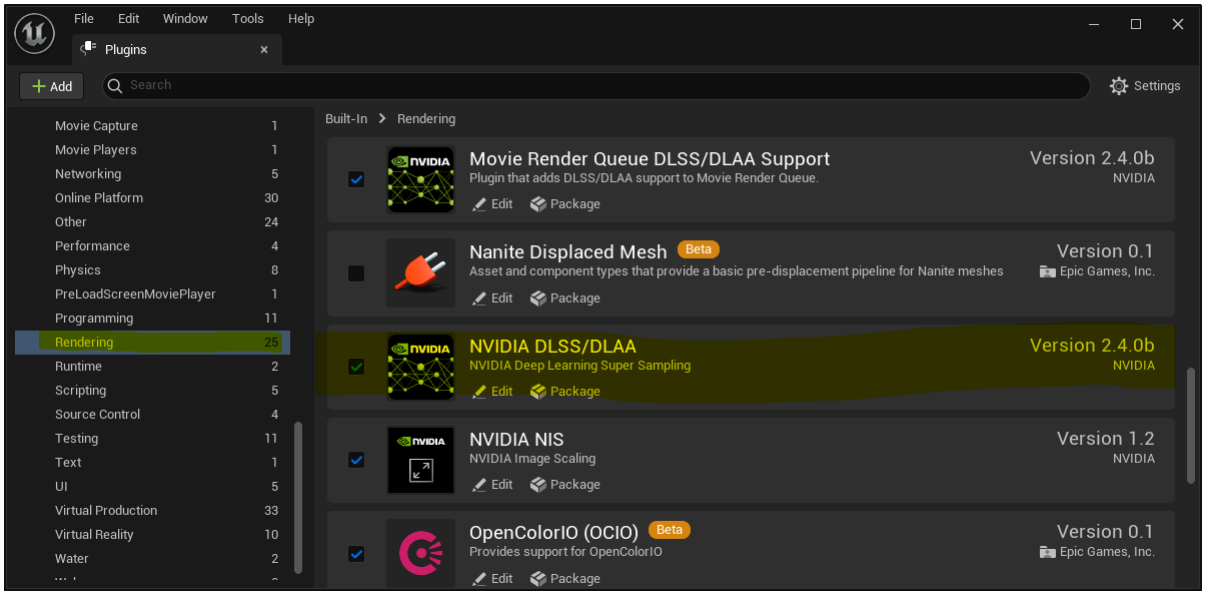
DLSS and the NIS NVIDIA Image Scaling plugin

The *DLSS* plugin and NVIDIA Image Scaling (*NIS*) plugins can be enabled together for the same project. Please see the `RTX UI Developer Guidelines` document for suggested UI implementations.

It is recommend to use the NIS plugin for sharpening in place of the deprecated DLSS sharpening feature.

DLSS/DLAA in the Editor

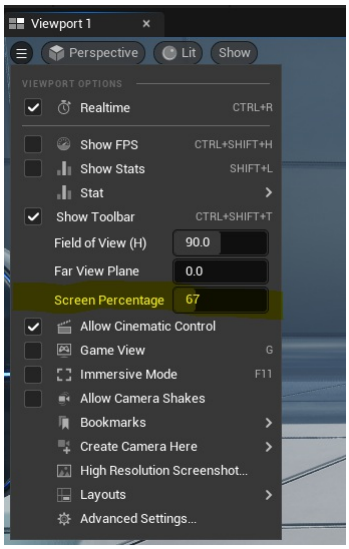
Enabling DLSS/DLAA for a project



Enabling DLSS/DLAA in Level Editor Viewports

First enable the "Enable DLSS/DLAA to be turned on in Editor viewports" checkbox in the project plugin settings `Project Settings -> Plugins -> NVIDIA DLSS` or the local override `Project Settings -> Plugins -> NVIDIA DLSS (Local)`. Note that DLSS/DLAA is off by default in editor viewport windows.

With DLSS/DLAA enabled, change the Screen Percentage in an editor viewport window to control the upscaling amount. With a screen percentage of 100, DLAA will be in effect. Sliding the screen percentage between 50% and 99% will enable DLSS upscaling.

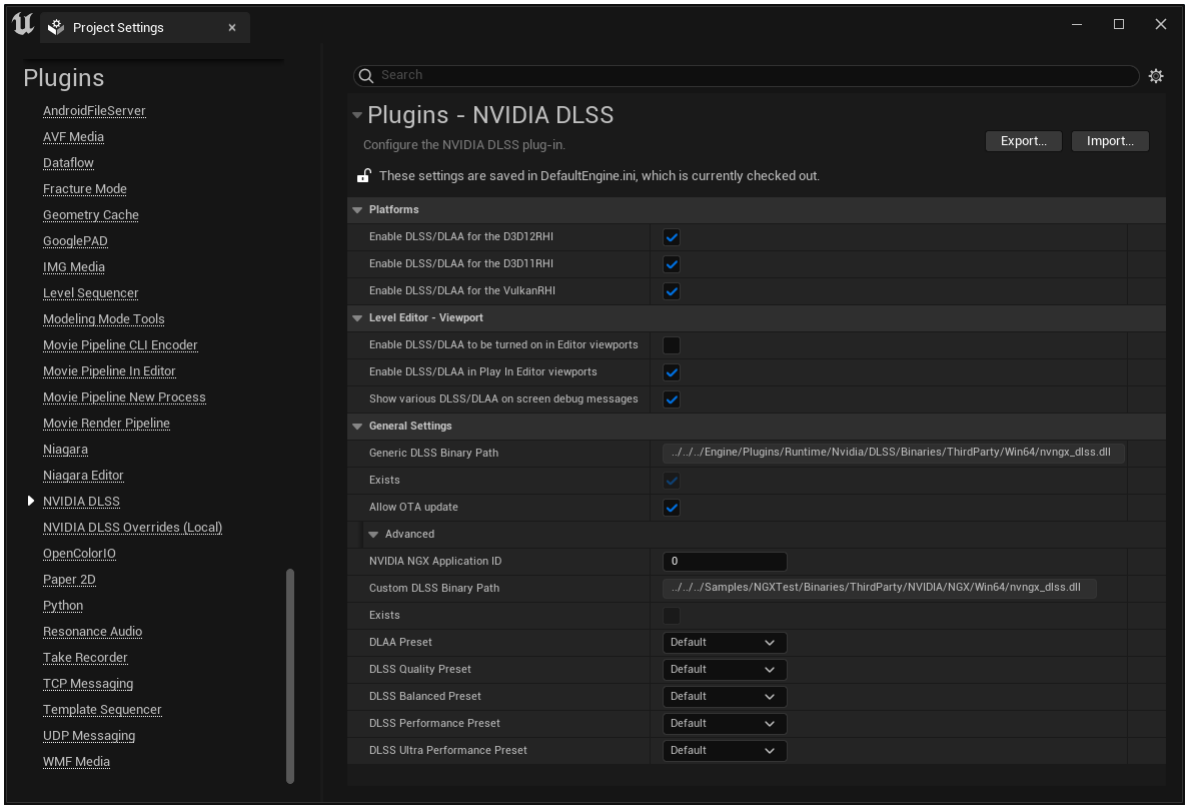


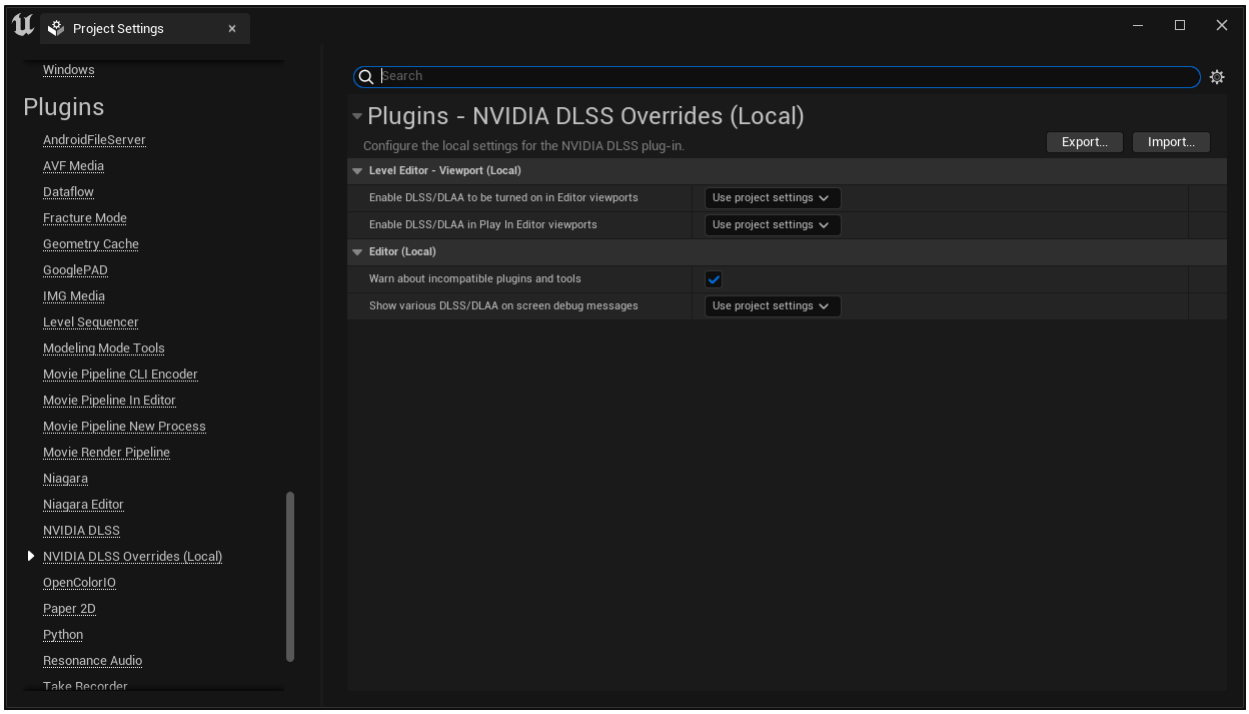
DLSS Plugin Settings

Some of the "Level Editor - Viewport" settings are split across two config files and settings pages to tailor how DLSS is interacting with the editor user experience.

For example, a cross-platform game project might find it more practical by default to only have DLSS enabled in "Play In Editor Viewports" or in "game mode" in order to maintain a consistent content authoring experience across the range of supported platforms. However projects (e.g. an architecture visualization project with notable raytracing workloads), might find it more useful to have DLSS enabled during the content authoring. Either way each user can override those settings locally:

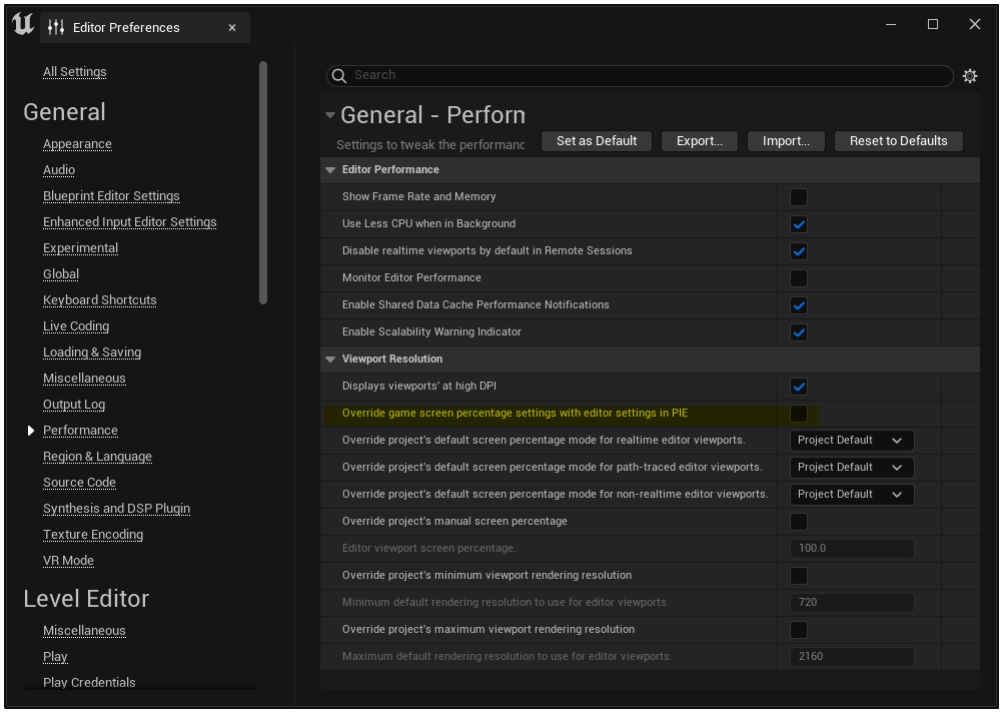
- `Project Settings -> Plugins -> NVIDIA DLSS`
  - stored in DefaultEngine.ini
  - typically resides in source control.
  - settings here are shared between users
- `Project Settings -> Plugins -> NVIDIA DLSS (Local)`
  - stored UserEngine.ini
  - not recommended to be checked into source control.
  - allow a user to override project wide settings if desired. Defaults to "use project settings"





Editor Settings

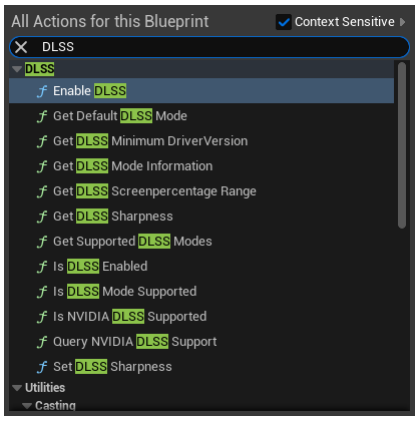
By default, the editor overrides the screen percentage when running in PIE. This can interfere with application logic that changes the screen percentage to set a particular DLSS quality level. To allow the application to set screen percentage, go to `Edit -> Editor Preferences -> Performance` and turn off "Override game screen percentage settings with editor settings in PIE". Restart the editor after changing this setting.



DLSS Blueprints

The UDLSSLibrary blueprint library provides functionality to query whether DLSS and which modes are supported. It also provides convenient functions to enable the underlying DLSS console variables. The tooltips of each function provide additional information.

Using the UDLSSLibrary via blueprint or C++ (by including the DLSSBlueprint module in a game project) is recommended over setting the console variables directly. This will make sure that any future updates will be picked up by simply updating the DLSS plugin, without having to update the game logic.

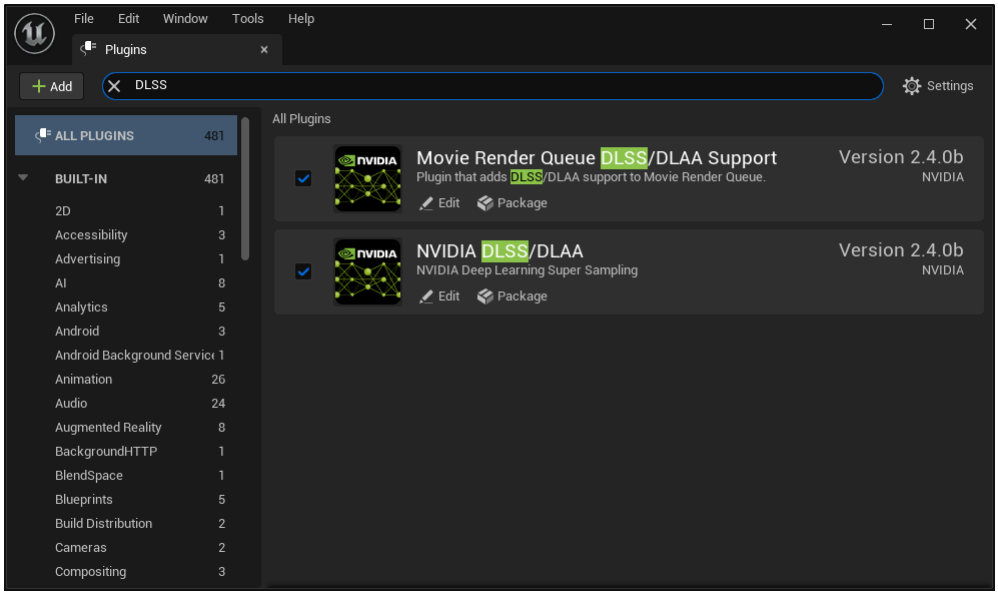


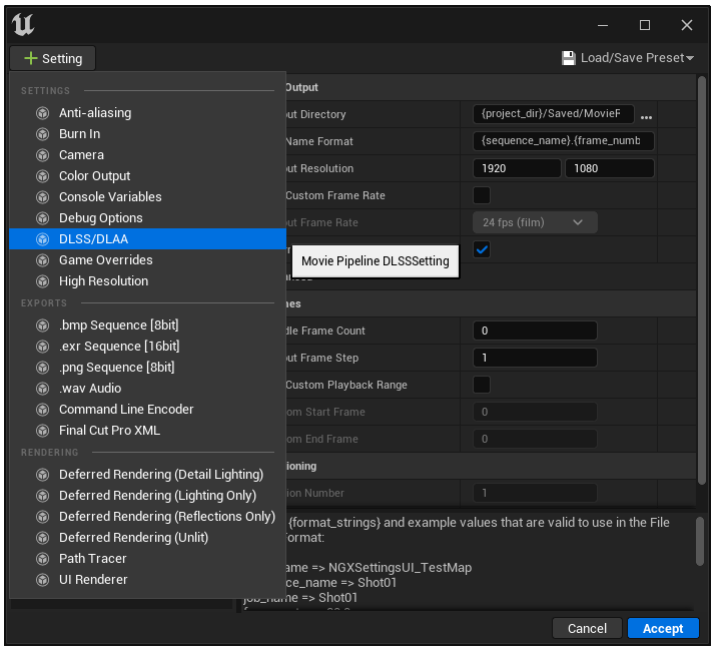
DLSS Movie Render Queue Support

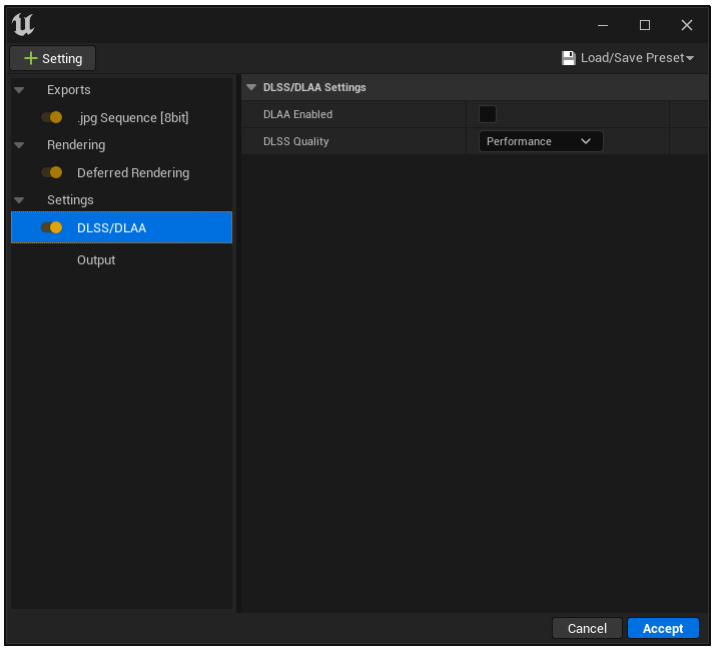
DLSS is supported when rendering movies with the Movie Render Queue plugin.

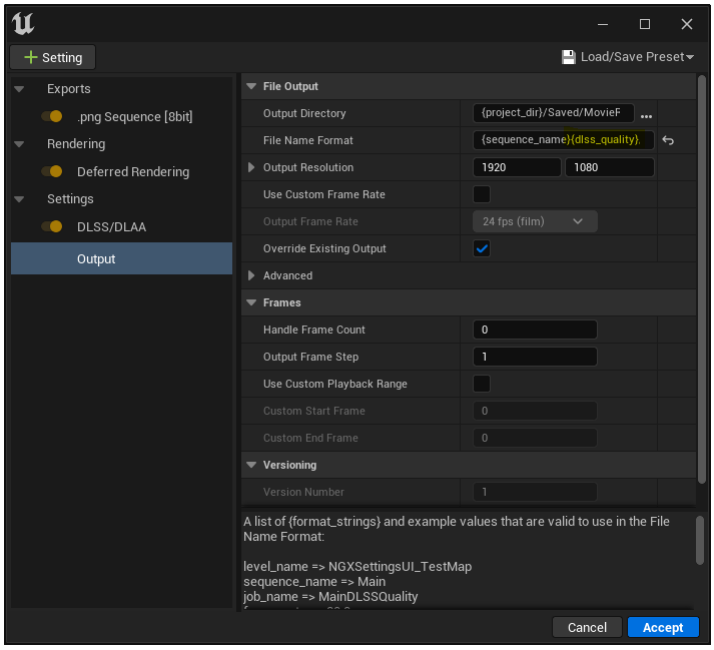
1. Enable the *Movie Render Queue* and *DLSS* plugins in the Editor
2. Enable the *Movie Render Queue DLSS Support* plugin in the Editor, then restart the editor
3. In the configuration, add the `Settings -> DLSS` page
4. In the DLSS settings page, change the desired DLSS quality mode
    1. Note: Unsupported DLSS modes will show a warning at the bottom of the window
5. Optional: The `Settings -> Output -> File Name Format` page supports a `{dlss_quality}` format tag

Note: Only the *Deferred Rendering* render pass is supported with DLSS, all other passes use the built-in TAA









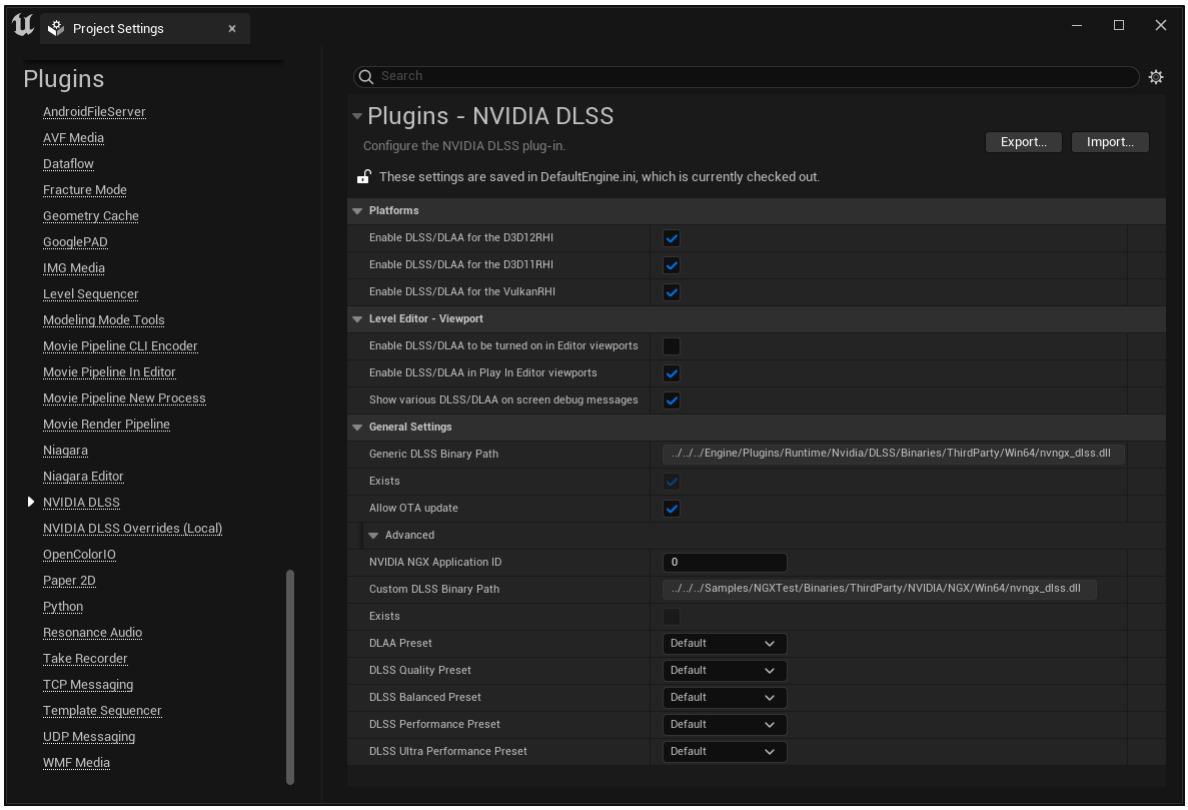
Distributing DLSS

The DLSS plugin ships with a ready-to-use production DLSS binary (without watermarks) and uses the project identifier to initialize NGX and DLSS. This is the common case for distribution to end users and does not require further actions from either your or NVIDIA's side. On rare occasion NVIDIA however might provide:

1. a custom project specific DLSS binary
2. an NVIDIA application ID

In that case those can be configured in the advanced plugin settings. Additionally please also ensure that the r.NGX.ProjectIdentifier console variable is set to either 0 (the default) or 2. The project plugin settings can be used to configure those (please see above).

1. The custom, project specific DLSS binary `nvngx_dlss.dll` should be put into the project under `$(ProjectDir)/Binaries/ThirdParty/NVIDIA/NGX/$(Platform)`
2. Setting the NVIDIA NGX application ID for the project.



Please refer to "Chapter 4 Distributing DLSS in a Game" in the the DLSS Programming Guide for details.

Upgrading from DLSS plugin versions for earlier engines

In general, it is strongly recommended to use the DLSS on-screen indicator when upgrading from engine versions before 5.1 to verify that you are still getting DLSS and DLAA enabled properly in your application.

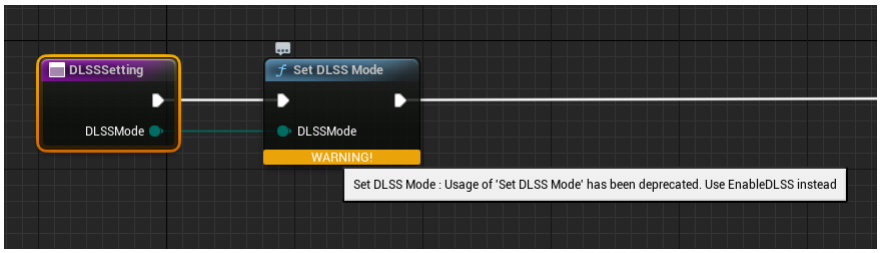
Screen percentage

Before Unreal Engine 5.1, the DLSS plugin controlled upscaling screen percentage directly, overriding anything set through the `r.ScreenPercentage` cvar or other methods. In Blueprints or in code, the application would request a particular DLSS quality mode from the plugin, and the plugin would internally set the quality mode and the appropriate screen percentage.

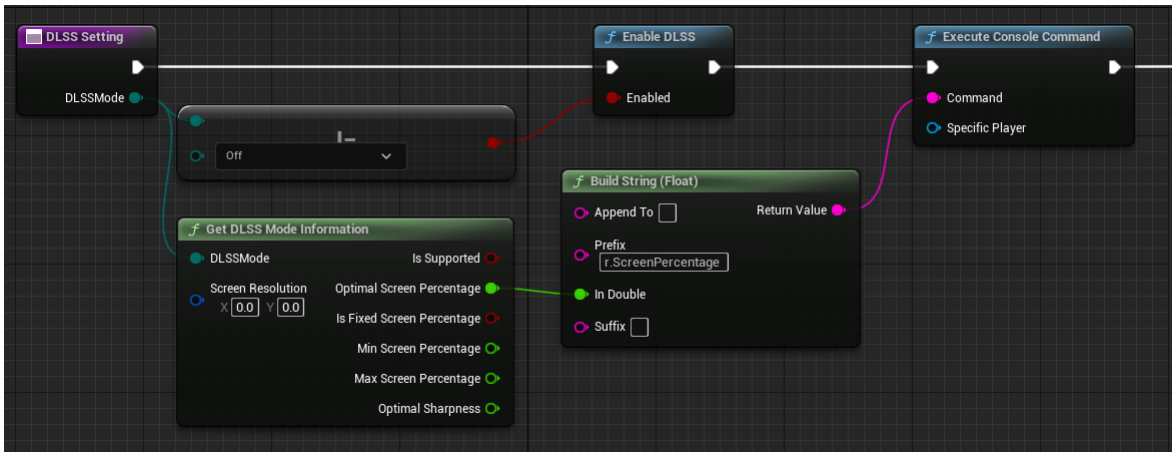
Starting with Unreal Engine 5.1, plugins do not override the upscaling screen percentage. So now to get a particular DLSS quality mode, the application must set the screen percentage corresponding to the optimal value for the desired DLSS quality mode. If DLSS is enabled, the plugin will internally set the best quality mode for the current screen percentage. GetDLSSModeInformation can be used from blueprints or from C++ to find the optimal screen percentage for a given DLSS quality mode. For DLAA, set a screen percentage of 100%.

The old SetDLSSMode and EnableDLAA blueprint functions have been deprecated and should be replaced with EnableDLSS, and then the appropriate screen percentage can be set by executing the `r.ScreenPercentage` console command from blueprints or code.

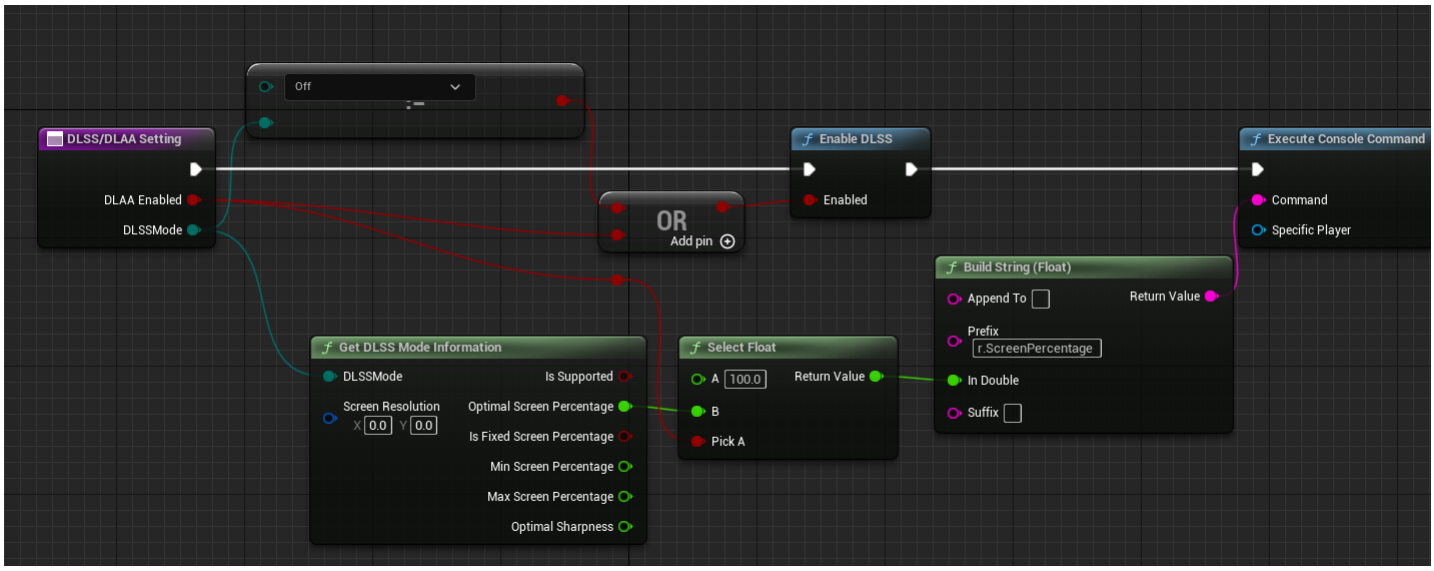
Before updating, you will see deprecation warnings for the old blueprint functions.



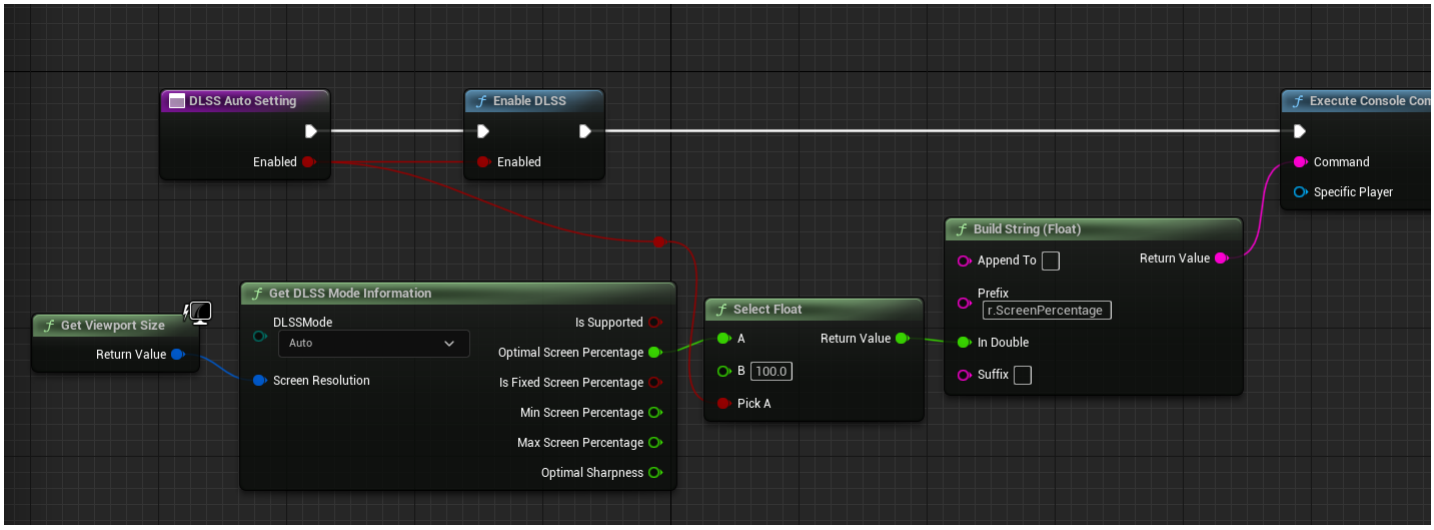
To offer the same DLSS quality mode options as before, check the optimal upscaling screen percentage for a quality mode and use the "r.ScreenPercentage" console command to set it.



DLAA is enabled by setting screen percentage to 100.



If you want your application to offer DLSS Off/Auto options instead of selecting individual DLSS quality modes, you can supply the screen resolution along with the fake "Auto" quality mode to GetDLSSModeInformation and a recommended screen percentage will be provided.



If your application changed `r.ScreenPercentage` before upgrading to Unreal Engine 5.1 or later, you may find that DLSS/DLAA behaves differently after upgrading. This is because before engine version 5.1, the DLSS plugin was able to override the screen percentage value when DLSS or DLAA were enabled. When using the DLSS on-screen indicator, if you see that the upscale resolution doesn't change when changing DLSS modes, check for any blueprints or code that may be setting `r.ScreenPercentage` directly.

Performance benefits of DLSS-SR

- Unreal Engine 5 scales the input resolution of the rendered scene automatically according to the current output resolution, ranging from 50 to 100%.
- Since all upscalers use the same automatic input resolution, performance should be similar across the available options.

- DLSS-SR has the advantage of delivering the best overall image quality thanks to the AI and deep learning techniques used to upscale and enhance the output. As a result, DLSS-SR has the ability to support resolutions below 50% with a 33% input resolution "Ultra Performance" mode to deliver significant performance gains while still preserving image quality.

Performance benefits of DLSS-RR

- When DLSS-RR is engaged, it effectively takes over super resolution and adds the benefit of replacing all available ray tracing denoisers with a single pass. This means, the more ray tracing technologies available, such as reflections and shadows, the more performance the app will receive when being replaced with DLSS-RR.
- That said, DLSS-RR provides the most benefit when in use with hardware ray tracing (HWRT). Raster (non-rt) and software ray tracing paths *may* experience an image quality uplift, entirely dependent on the project. However, it will likely be slight and the performance will either remain about the same or be slightly impacted.
- If using the `Is DLSS-RR Supported` blueprint function, that will automatically determine if HWRT is available. If it is not available, DLSS-RR will come back as unsupported.

DLSS API and UI Documentation

The DLSS Programming Guide provides details about the NVIDIA NGX APIs which are used by the plugin to implement DLSS.

The RTX UI Developer Guidelines (Chinese) provides details about recommended game settings and UI for DLSS.

The NVIDIA Developer Blog Tips: Getting the Most out of the DLSS Unreal Engine 4 Plugin provides best practices along with other tips and tricks to use NVIDIA DLSS in Unreal Engine games and applications.

Graphics Debug and Validation Layers

The UE plugins connect the various UE RHIs, the underlying graphics APIs and NVIDIA SDKs. As such they are exposed to changes across RHIs across UE versions.

The underlying hard and software stack is pretty indifferent to the resource states of modern graphics APIs, but consistent resource state between the engine/RHI, the graphics API and the NVIDIA SDK's is needed for compatibility with graphics API debug layers.

This section provides an overview over those parts of the UE plugins, please refer to the source and comments for further details.

UE 5.6 and newer

UE 5.6:

- Moved the majority of the resource state management from the D3D12RHI to the RDG, removing the D3D12RHI resource state tracker.
- Adds `ID3D12DynamicRHI::RHIUpdateResourceResidency` that allows plugins to make input and output resources resident.
  - This avoids GPU memory access faults when the system is under memory pressure.
- Adds `ID3D12DynamicRHI::RHIFlushResourceBarriers` that allows plugins to flush pending resource barriers of the D3D12RHI to be submitted to the command list.
  - This allows D3D12 resource states match between the RHI, the DX12 API and the NVIDIA SDK, thus avoiding D3D debug layer error.

UE 5.5 and older

UE 5.5 and older:

- Has resource state management split between the RDG and the D3D12RHI resource state tracker.
- Do not support `ID3D12DynamicRHI::RHIUpdateResourceResidency`.
  - In 5.5 particular this can lead to lost devices / GPU memory fault when the system is under memory pressure because the plugin resources are not made D3D12 resident before calling into the NVIDIA SDK.
- Do not support `ID3D12DynamicRHI::RHIFlushResourceBarriers`.
  - This means that it's impossible to pass the actual D3D12 API resource state correctly into the NVIDIA SDKs. This then causes d3d debug validation layer messages.

However the effects (residency management and pending barrier flushing) of those UE 5.6 `ID3D12DynamicRHI` methods can be achieved by ~~abuses~~ side-effects of existing UE 5.5 `ID3D12DynamicRHI` methods that call the D3D12RHI side internal methods that are exposed directly via the 5.6 methods:

- `ID3D12DynamicRHI::RHIUpdateResourceResidency` can be achieved by calling UE 5.5 `ID3D12DynamicRHI::RHITransitionResource` on the plugin input and output resources
  - Calling those methods also puts the resources into a consistent D3D12 resource state at the RHI level that then can be passed into the NVIDIA SDK's. However those resource transitions are pending at the D3D12RHI and need to be flushed, as discussed below.
- `ID3D12DynamicRHI::RHIFlushResourceBarriers` can be achieved by calling UE 5.5 `RHICopyTexture` on unrelated textures.
  - This creates additional RDG passes and resources that have resource labels and draw event names like

`UE5.5AndOlderDebugLayerCompatibilityHelperSource` or `UE5_5AndOlderBackdoorViaUnrelatedCopy` that should make them easy to spot in debugging tools such as Nsight Graphics
  - Since this is only needed for D3D debug layer compatibility and not functionality, those extra passes are only executed when the d3d debug layer is enabled by the engine, either via `-d3ddebug` or the respective config file section.
  - **Note:** `-d3ddebug` is also supported in `UE_BUILD_SHIPPING`, and so are the additional debug layer compatibility passes.

Experimental Backports from 5.6 `ID3D12DynamicRHI` methods to 5.5 and older

If those trade-offs from above are insufficient for a particular use case, the `ID3D12DynamicRHI::RHIFlushResourceBarriers` and `ID3D12DynamicRHI::RHIUpdateResourceResidency` are isolated enough that they could be backported to 5.5 and older if needed.

1. Cherry-pick `2be94f9e1642ca026c07cbcc980ac922afc17b00 Add RHIUpdateResourceResidency API to ID3D12DynamicRHI`
2. Cherry-pick `642d7f4108155528627f27eec161f06936d97659 Allow DX12 Resource barrier flushing trough ID3D12DynamicRHI`
3. Add to the bottom of `RHIDefinitions.h` (and not `ID3D12DynamicRHI.h`): `#define ENGINE_PROVIDES_UE_5_6_ID3D12DYNAMICRHI_METHODS 1`
4. The UE plugin guards relevant code paths by `ENGINE_PROVIDES_UE_5_6_ID3D12DYNAMICRHI_METHODS`

Note: This is path is provided as-is and might require adjustments